

Авторские права
© Postgres Professional, 2017–2024
Авторы: Егор Рогов, Павел Лузанов, Илья Баштанов, Игорь Гнатюк
Фото: Олег Бартунов (монастырь Пху и пик Бхрикути, Непал)

Использование материалов курса
Некоммерческое использование материалов курса (презентации, демонстрации) разрешается без ограничений. Коммерческое использование возможно только с письменного разрешения компании Postgres Professional. Запрещается внесение изменений в материалы курса.

Обратная связь
Отзывы, замечания и предложения направляйте по адресу:
edu@postgrespro.ru

Отказ от ответственности
Компания Postgres Professional не несет никакой ответственности за любые повреждения и убытки, включая потерю дохода, нанесенные прямым или непрямым, специальным или случайным использованием материалов курса. Компания Postgres Professional не предоставляет каких-либо гарантий на материалы курса. Материалы курса предоставляются на основе принципа «как есть» и компания Postgres Professional не обязана предоставлять сопровождение, поддержку, обновления, расширения и изменения.

Темы

Процедуры и их отличие от функций

Входные и выходные параметры

Перегрузка и полиморфизм

Подпрограммы

Функции

- вызываются в контексте выражения
- не могут управлять транзакциями
- возвращают результат

Процедуры

- вызываются оператором CALL
- могут управлять транзакциями
- могут возвращать результат

Процедуры были введены в PostgreSQL 11. Основная причина их появления состоит в том, что функции не могут управлять транзакциями. Функции вызываются в контексте какого-либо выражения, которое вычисляется как часть уже начатого оператора (например, SELECT) в уже начатой транзакции. Нельзя завершить транзакцию и начать новую «посередине» выполнения оператора.

Процедуры всегда вызываются специальным оператором CALL. Если этот оператор сам начинает новую транзакцию (а не вызывается из уже начатой), то в процедуре можно использовать команды управления транзакциями.

К сожалению, процедуры, написанные на языке SQL, лишены сейчас возможности использовать команды COMMIT и ROLLBACK, хотя те из них, что оформлены в новом стиле стандарта SQL, вероятно смогут это делать в будущем. Поэтому пример процедуры, управляющей транзакциями, придется отложить до темы «PL/pgSQL. Выполнение запросов».

Иногда можно услышать, что процедура отличается от функции тем, что не возвращает результат. Но это не так — процедуры тоже могут возвращать результат, если необходимо.

Функции и процедуры имеют общее пространство имен и вместе называются *подпрограммами (routine)*.

https://postgrespro.ru/docs/postgresql/16/sql-createprocedure

https://postgrespro.ru/docs/postgresql/16/sql-call

Процедуры без параметров

Начнем с примера простой процедуры без параметров.

```
=> CREATE TABLE t(a float);
```

```
CREATE TABLE
```

```
=> CREATE PROCEDURE fill()
AS $$
    TRUNCATE t;
    INSERT INTO t SELECT random() FROM generate_series(1,3);
$$ LANGUAGE sql;
```

```
CREATE PROCEDURE
```

Чтобы вызвать процедуру, необходимо использовать специальный оператор:

```
=> CALL fill();
```

```
CALL
```

Результат работы виден в таблице:

```
=> SELECT * FROM t;
```

```
          a
---------------------
  0.9496231769169448
  0.9576499851047511
 0.45647389772638647
(3 rows)
```

А теперь переопределим нашу процедуру в стиле стандарта SQL:

```
=> CREATE OR REPLACE PROCEDURE fill()
LANGUAGE sql
BEGIN ATOMIC
    DELETE FROM t;  -- команда TRUNCATE пока что не поддерживается в таких подпрограммах
    INSERT INTO t SELECT random() FROM generate_series(1,3);
END;
```

```
CREATE PROCEDURE
```

Убедимся в ее работоспособности:

```
=> CALL fill();
```

```
CALL
```

```
=> SELECT * FROM t;
```

```
          a
---------------------
  0.5143406097651662
 0.23528484532400729
  0.9093765558795717
(3 rows)
```

И попробуем в процедуре выполнить фиксацию транзакции:

```
=> CREATE OR REPLACE PROCEDURE fill()
LANGUAGE sql
BEGIN ATOMIC
    DELETE FROM t;
    INSERT INTO t SELECT random() FROM generate_series(1,3);
    COMMIT;
END;
```

```
ERROR:  COMMIT is not yet supported in unquoted SQL function body
```

Обратите внимание, что мы получили ошибку о недопустимой команде еще на этапе определения подпрограммы.

Переименуем таблицу, с которой работает наша процедура:

```
=> ALTER TABLE t RENAME TO ta;
```

```
ALTER TABLE
```

Вызов ниже не приведет к ошибке — таблица в определении процедуры в системном каталоге теперь представлена не по имени, а по идентификатору, который был получен еще на этапе создания подпрограммы.

```
=> CALL fill();
```

```
CALL
```

Ту же самую задачу, что выполняла процедура, можно решить и с помощью функции, возвращаемое значение которой определяется последним оператором. Можно объявить тип результата void, если фактически функция ничего не возвращает, или вернуть что-то осмысленное.

Возвратим таблице прежнее имя и определим функцию:

```
=> ALTER TABLE ta RENAME TO t;
```

```
ALTER TABLE
```

```
=> CREATE FUNCTION fill_avg() RETURNS float
LANGUAGE sql
BEGIN ATOMIC
    DELETE FROM t;
    INSERT INTO t SELECT random() FROM generate_series(1, 3);
    SELECT avg(a) FROM t;
END;
```

```
CREATE FUNCTION
```

В любом случае функция вызывается в контексте какого-либо выражения:

```
=> SELECT fill_avg();
```

```
      fill_avg
--------------------
 0.3006583644094014
(1 row)


=> SELECT * FROM t;

          a
----------------------
   0.5732569517817321
   0.2756659879596799
 0.053052153486792264
(3 rows)
```

Чего нельзя достичь с помощью функции — это управления транзакциями. Но и в процедурах на языке SQL, как мы видели, это пока не поддерживается (зато поддерживается при использовании других языков).

Процедуры с параметрами

Добавим в процедуру входной параметр — число строк:

```
=> DROP PROCEDURE fill();
```

```
DROP PROCEDURE
```

```
=> CREATE PROCEDURE fill(nrows integer)
LANGUAGE sql
BEGIN ATOMIC
    DELETE FROM t;
    INSERT INTO t SELECT random() FROM generate_series(1, nrows);
END;
```

```
CREATE PROCEDURE
```

Точно так же, как и в случае функций, при вызове процедур фактические параметры можно передавать позиционным способом или по имени:

```
=> CALL fill(nrows => 5);
```

```
CALL
```

```
=> SELECT * FROM t;
```

```
          a
--------------------
 0.21214735877191093
 0.18596723149308203
  0.3794369561139086
 0.24575229241754926
    0.80230107081726
(5 rows)
```

Процедуры могут также иметь OUT- и INOUT-параметры, с помощью которых можно возвращать значения:

```
=> DROP PROCEDURE fill(integer);
```

```
DROP PROCEDURE
```

```
=> CREATE PROCEDURE fill(IN nrows integer, OUT average float)
LANGUAGE sql
BEGIN ATOMIC
    DELETE FROM t;
    INSERT INTO t SELECT random() FROM generate_series(1, nrows);
    SELECT avg(a) FROM t; -- как в функции
END;
```

```
CREATE PROCEDURE
```

Попробуем:

```
=> CALL fill(5, NULL /* значение не используется, но его необходимо указать*/);
```

```
      average
--------------------
 0.6208802949059609
(1 row)
```

Перегрузка

Несколько подпрограмм с одним и тем же именем

- подпрограмма однозначно определяется сигнатурой — именем и входными параметрами
- тип возвращаемого значения и выходные параметры игнорируются
- подходящая подпрограмма выбирается во время выполнения в зависимости от фактических параметров

Команда CREATE OR REPLACE

- при несовпадении типов входных параметров создаст новую перегруженную подпрограмму
- при совпадении — изменит существующую подпрограмму, но нельзя поменять тип подпрограммы, тип возвращаемого значения, типы OUT-параметров

Перегрузка — это возможность использования одного и того же имени для нескольких подпрограмм, отличающихся типами параметров IN и INOUT.

Сигнатура подпрограммы — ее имя и типы входных параметров. При вызове PostgreSQL находит подпрограмму, соответствующую сигнатуре. Возможны ситуации, когда подходящую подпрограмму невозможно определить однозначно; в таком случае во время выполнения возникнет ошибка.

Однако в сигнатуру, в частности, не входят:

- тип подпрограммы (процедура или функция);
- типы параметров OUT;
- тип возвращаемого значения.

Перегрузку надо учитывать, выполняя команду CREATE OR REPLACE (FUNCTION или PROCEDURE). Если сигнатура не совпадает ни с одной из существующих, будет создана новая перегруженная подпрограмма, а при совпадении — изменена существующая.
В последнем случае не разрешается изменять тип подпрограммы, тип OUT-параметров или тип возвращаемого значения, однако можно изменить язык и другие свойства. Поэтому иногда приходится удалять подпрограмму и создавать ее заново, в этом случае потребуется также удалить зависящие от нее объекты: другие подпрограммы, представления, триггеры и т. п. (DROP ROUTINE ... CASCADE).

https://postgrespro.ru/docs/postgresql/16/xfunc-overload

Полиморфизм

Подпрограмма, принимающая параметры разных типов

- формальные параметры используют полиморфные псевдотипы (например, `anyelement` или `anycompatible`)
- конкретный тип данных выбирается во время выполнения по типу фактических параметров

В некоторых случаях удобно не создавать несколько перегруженных подпрограмм для разных типов, а написать одну, принимающую параметры любого (или почти любого) типа.

Для этого в качестве типа формального параметра указывается специальный *полиморфный псевдотип*. Пока мы рассмотрим два из них — anyelement и anycompatible — но позже встретимся и с другими.

Полиморфные псевдотипы в определении позволяют использовать в качестве параметров любые типы, при этом конкретный тип, с которым будет работать подпрограмма, выбирается во время выполнения по типу фактического параметра.

Если в определении указано несколько полиморфных параметров типа anyelement, то все фактические параметры неявно приводятся к типу первого из них. Если же указано несколько параметров типа anycompatible, то типы фактических параметров приводятся к некоторому общему типу.

Если подпрограмма объявлена с возвращаемым значением полиморфного типа, то она должна иметь по крайней мере один входной полиморфный параметр. Конкретный тип возвращаемого значения также определяется исходя из типа фактического входного параметра. Для подпрограмм в стиле стандарта SQL возможности использовать полиморфные типы данных для аргументов нет.

https://postgrespro.ru/docs/postgresql/16/extend-type-system#EXTEND-TYPES-POLYMORPHIC

https://postgrespro.ru/docs/postgresql/16/xfunc-sql#XFUNC-SQL-POLYMORPHIC-FUNCTIONS

Перегруженные подпрограммы

Перегрузка работает одинаково и для функций, и для процедур. Они имеют общее пространство имен.

В качестве примера напишем функцию, возвращающую большее из двух целых чисел. (Похожее выражение есть в SQL и называется greatest, но мы напишем собственную функцию.)

```
=> CREATE FUNCTION maximum(a integer, b integer) RETURNS integer
LANGUAGE sql
RETURN CASE WHEN a > b THEN a ELSE b END;
```

```
CREATE FUNCTION
```

Проверим:

```
=> SELECT maximum(10, 20);
```

```
 maximum
---------
      20
(1 row)
```

Допустим, мы решили сделать аналогичную функцию для трех чисел. Благодаря перегрузке, не надо придумывать для нее какое-то новое название:

```
=> CREATE FUNCTION maximum(a integer, b integer, c integer)
RETURNS integer
LANGUAGE sql
RETURN CASE
         WHEN a > b THEN maximum(a, c)
         ELSE maximum(b, c)
       END;
```

```
CREATE FUNCTION
```

Теперь у нас две функции с одним именем, но разным числом параметров:

```
=> \df maximum
```

```
                              List of functions
 Schema |  Name   | Result data type |       Argument data types       | Type
--------+---------+------------------+---------------------------------+------
 public | maximum | integer          | a integer, b integer            | func
 public | maximum | integer          | a integer, b integer, c integer | func
(2 rows)
```

И обе работают:

```
=> SELECT maximum(10, 20), maximum(10, 20, 30);
```

```
 maximum | maximum
---------+---------
      20 |      30
(1 row)
```

Команда CREATE OR REPLACE позволяет создать подпрограмму или заменить существующую, не удаляя ее. Поскольку в данном случае функция с такой сигнатурой уже существует, она будет заменена:

```
=> CREATE OR REPLACE FUNCTION maximum(a integer, b integer, c integer)
RETURNS integer
LANGUAGE sql
RETURN CASE
         WHEN a > b THEN
           CASE WHEN a > c THEN a ELSE c END
         ELSE
           CASE WHEN b > c THEN b ELSE c END
         END;
```

```
CREATE FUNCTION
```

Пусть наша функция работает не только для целых чисел, но и для вещественных. Как этого добиться? Можно определить еще такую функцию:

```
=> CREATE FUNCTION maximum(a real, b real) RETURNS real
LANGUAGE sql
RETURN CASE WHEN a > b THEN a ELSE b END;
```

```
CREATE FUNCTION
```

Теперь у нас три функции с одинаковым именем:

```
=> \df maximum

                              List of functions
 Schema |  Name   | Result data type |       Argument data types       | Type
--------+---------+------------------+---------------------------------+------
 public | maximum | integer          | a integer, b integer            | func
 public | maximum | integer          | a integer, b integer, c integer | func
 public | maximum | real             | a real, b real                  | func
(3 rows)
```

Две из них имеют одинаковое количество параметров, но отличаются их типами:

```
=> SELECT maximum(10, 20), maximum(1.1, 2.2);

 maximum | maximum
---------+---------
      20 |     2.2
(1 row)
```

Если подпрограмма перегружена несколько раз, то для получения информации только о некоторых из них можно указать в команде \df типы интересующих параметров:

```
=> \df maximum real

                        List of functions
 Schema |  Name   | Result data type | Argument data types | Type
--------+---------+------------------+---------------------+------
 public | maximum | real             | a real, b real      | func
(1 row)
```

Дальше нам придется определить функции для всех остальных типов данных и повторить все то же самое для трех параметров. Притом что операторы в теле этих функций будут одни и те же.

Полиморфные функции

Здесь нам помогут полиморфные типы anyelement и anycompatible. Это псевдотипы, взамен которых при вызове и интерпретации функции будет подставлен тип фактического параметра. Разумеется, в случае определения подпрограммы в стиле стандарта SQL ее код будет разобран еще на этапе создания, и воспользоваться полиморфными псевдотипами не удастся.

Удалим все три наши функции...

```
=> DROP FUNCTION maximum(integer, integer);

DROP FUNCTION

=> DROP FUNCTION maximum(integer, integer, integer);

DROP FUNCTION

=> DROP FUNCTION maximum(real, real);

DROP FUNCTION
```

...и затем создадим новую:

```
=> CREATE FUNCTION maximum(a anyelement, b anyelement)
RETURNS anyelement
AS $$
    SELECT CASE WHEN a > b THEN a ELSE b END;
$$ LANGUAGE sql;

CREATE FUNCTION
```

Такая функция должна принимать любой тип данных (а работать будет с любым типом, для которого определен оператор «больше»).

Получится?

```
=> SELECT maximum('A', 'B');

ERROR:  could not determine polymorphic type because input has type unknown
```

Увы, нет. В данном случае строковые литералы могут быть типа char, varchar, text — конкретный тип нам неизвестен. Но можно применить явное приведение типов:

```
=> SELECT maximum('A'::text, 'B'::text);

 maximum
---------
 B
(1 row)
```

Еще пример с другим типом:

```
=> SELECT maximum(now(), now() + interval '1 day');

            maximum
-------------------------------
 2025-06-25 04:33:00.367568+03
(1 row)
```

Тип результата функции всегда будет тот же, что и тип параметров.

Но можно продвинуться еще дальше — сделать так, чтобы можно было использовать в полиморфных подпрограммах не абсолютно одинаковые типы, а совместимые, то есть те, что могут быть приведены друг к другу неявно. Для этого нужно использовать полиморфный псевдотип anycompatible.

Удалим нашу функцию и взамен нее создадим другую:

```
=> DROP FUNCTION maximum;

DROP FUNCTION

=> CREATE FUNCTION maximum(a anycompatible, b anycompatible)
RETURNS anycompatible
AS $$
    SELECT CASE WHEN a > b THEN a ELSE b END;
$$ LANGUAGE sql;

CREATE FUNCTION
```

Повторим наш опыт с параметрами-литералами:

```
=> SELECT maximum('A', 'B');

 maximum
---------
 B
(1 row)
```

Получилось!

Но если типы параметров не совпадают и не могут быть неявно приведены к некоторому общему типу, то будет ошибка:

```
=> SELECT maximum(1, 'A');

ERROR:  invalid input syntax for type integer: "A"
LINE 1: SELECT maximum(1, 'A');
                          ^
```

В этом примере такое ограничение выглядит естественно, хотя в некоторых случаях оно может оказаться и неудобным.

Определим теперь функцию с тремя параметрами, но так, чтобы третий можно было не указывать:

```
=> CREATE FUNCTION maximum(
    a anycompatible,
    b anycompatible,
    c anycompatible DEFAULT NULL
) RETURNS anycompatible
AS $$
SELECT CASE
         WHEN c IS NULL THEN
             x
         ELSE
             CASE WHEN x > c THEN x ELSE c END
       END
FROM (
    SELECT CASE WHEN a > b THEN a ELSE b END
) max2(x);
$$ LANGUAGE sql;

CREATE FUNCTION

=> SELECT maximum(10, 11.21, 3e3);
```

```
 maximum
---------
    3000
(1 row)
```

Так работает. А так?

```
=> SELECT maximum(10, 11.21);
```

```
ERROR:  function maximum(integer, numeric) is not unique
LINE 1: SELECT maximum(10, 11.21);
               ^
HINT:  Could not choose a best candidate function. You might need to add explicit type
casts.
```

А так произошел конфликт перегруженных функций:

```
=> \df maximum
```

```
                                                 List of functions
 Schema |  Name   | Result data type |                              Argument data types
                    | Type
--------+---------+------------------+-----------------------------------------------------
---------------------+------
 public | maximum | anycompatible    | a anycompatible, b anycompatible
                    | func
 public | maximum | anycompatible    | a anycompatible, b anycompatible, c anycompatible
DEFAULT NULL::unknown | func
(2 rows)
```

Невозможно понять, имеем ли мы в виду функцию с двумя параметрами, или с тремя (но просто не указали последний).

Мы решим этот конфликт просто — удалим первую функцию за ненадобностью.

```
=> DROP FUNCTION maximum(anycompatible, anycompatible);
```

```
DROP FUNCTION
```

```
=> SELECT maximum(10, 11.21), maximum(10, 11.21, 3e3);
```

```
 maximum | maximum
---------+---------
   11.21 |    3000
(1 row)
```

Теперь все работает. А в теме «PL/pgSQL. Массивы» мы узнаем, как определять подпрограммы с произвольным числом параметров.

Итоги

Можно создавать и использовать собственные процедуры

В отличие от функций, процедуры вызываются оператором CALL и могут управлять транзакциями

Для процедур и функций поддерживаются перегрузка и полиморфизм